



Hands-on Lab Session

Class 07

Barry Grant

UC San Diego

<http://thegrantlab.org>

datacamp

HomeLearnCertificationSandboxJobsGroupsDataLab

S25_bioinformaticsClassroom

Invite Members

DashboardMembersTeamsSettings

LEARN

Custom TracksAssignmentsLeaderboard

INSIGHTS & ANALYTICS

ReportingSkill Matrix

CERTIFICATION

OverviewCertify My Team

PRO SERVICES

Live Trainings

app.datacamp.com/groups/25_bioinformatics/assignments

HomeGmailGcalGitHubBIMM143BGGN213BGGN239GDriveAtmosphereBIMM194BlinkNewsManage chats | LM Studio DocsBIMM 143 · An introductory hands-on course in Bioinfo...AssignmentsReview Answers | Gradescope

Assignments / Everyone

Create Assignment

ACTIVEPAST DUEARCHIVED

Active Assignments

Filter By Type

Search assignments...

TITLE	ASSIGNED TO	STATUS	DUE DATE TYPE	CREATED AT	EXPIRES AT	C	A	CR	DETAILS
 Introduction to Data Visualization with ggplot2 Course	Organization	Active	Fixed	Apr 16, 10:54 PDT	Apr 28, 12:00 PDT	8	41	19%	<button>View</button>
 Intermediate R Functions Chapter	Organization	Active	Fixed	Apr 22, 08:59 PDT	May 5, 12:00 PDT	0	0	0%	<button>View</button>

app.datacamp.com/groups/25_bioinformatics/assignments

datacamp Home Learn Certification Sandbox Jobs **Groups** DataLab

S25_bioinformatics Classroom

Assignments / Everyone

Create Assignment

ACTIVE PAST DUE ARCHIVED

Active Assignments

Filter By Type

Search assignments...

TITLE	ASSIGNED TO	STATUS	DUE DATE TYPE	CREATED AT	EXPIRES AT	C	A	CR	DETAILS
Introduction to Data Visualization with ggplot2 Course	Organization	Active	Fixed	Apr 16, 10:54 PDT	Apr 28, 12:00 PDT	8	41	19%	View
Intermediate R Functions Chapter	Organization	Active	Fixed	Apr 22, 08:59 PDT	May 5, 12:00 PDT	0	0	0%	View

LEARN Custom Tracks Assignments Leaderboard

INSIGHTS & ANALYTICS Reporting Skill Matrix

CERTIFICATION Overview Certify My Team

PRO SERVICES NEW Live Trainings

Last class!

Last class revisited...

function()

And some function homework....

Complete Q6. In last days lab supplement

```
library(bio3d)
s1 <- read.pdb("4AKE") # kinase with drug
s2 <- read.pdb("1AKE") # kinase no drug
s3 <- read.pdb("1E4Y") # kinase with drug

s1.chainA <- trim.pdb(s1, chain="A", elety="CA")
s2.chainA <- trim.pdb(s2, chain="A", elety="CA")
s3.chainA <- trim.pdb(s1, chain="A", elety="CA")

s1.b <- s1.chainA$atom$b
s2.b <- s2.chainA$atom$b
s3.b <- s3.chainA$atom$b

plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
plotb3(s2.b, sse=s2.chainA, typ="l", ylab="Bfactor")
plotb3(s3.b, sse=s3.chainA, typ="l", ylab="Bfactor")
```

Recap From Last Time:

→ **How:** Follow a step-by-step procedure to go from working code snippet to refined and tested function.

1. Start with a simple problem and write a working snippet of code.
2. Rewrite for clarity and to reduce duplication
3. Then, and only then, turn into an initial function
4. Test on small well defined input
5. Report on potential problem by failing early and loudly!

...

```
library(bio3d)
s1 <- read.pdb("4AKE") # kinase with drug
s2 <- read.pdb("1AKE") # kinase no drug
s3 <- read.pdb("1E4Y") # kinase with drug

s1.chainA <- trim.pdb(s1, chain="A", elety="CA")
s2.chainA <- trim.pdb(s2, chain="A", elety="CA")
s3.chainA <- trim.pdb(s1, chain="A", elety="CA")

s1.b <- s1.chainA$atom$b
s2.b <- s2.chainA$atom$b
s3.b <- s3.chainA$atom$b

plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
plotb3(s2.b, sse=s2.chainA, typ="l", ylab="Bfactor")
plotb3(s3.b, sse=s3.chainA, typ="l", ylab="Bfactor")
```

```
library(bio3d)
s1 <- read.pdb("4AKE") # kinase with drug
s2 <- read.pdb("1AKE")
s3 <- read.pdb("1E4Y")

s1.chainA <- trim.pdb(s1, chain="A", elety="CA")
s2.chainA <- trim.pdb(s2, chain="A", elety="CA")
s3.chainA <- trim.pdb(s1, chain="A", elety="CA")

s1.b <- s1.chainA$atom$b
s2.b <- s2.chainA$atom$b
s3.b <- s3.chainA$atom$b

plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
plotb3(s2.b, sse=s2.chainA, typ="l", ylab="Bfactor")
plotb3(s3.b, sse=s3.chainA, typ="l", ylab="Bfactor")
```

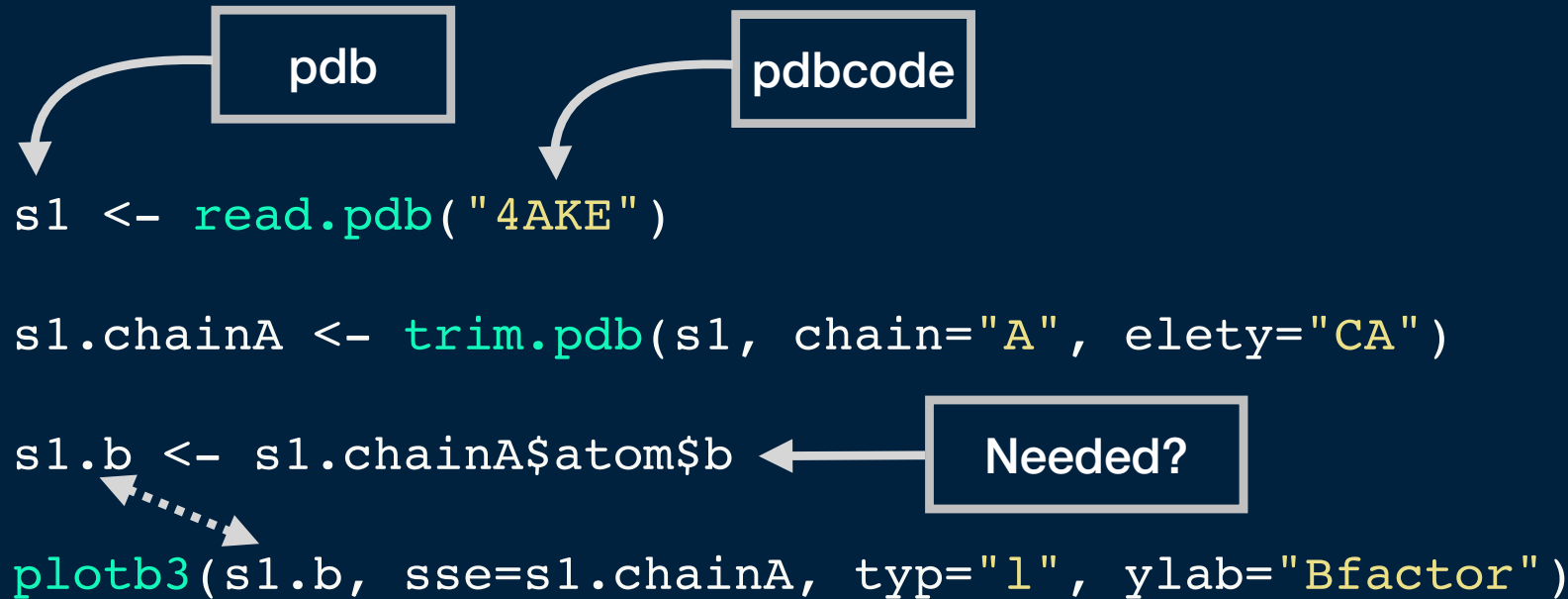
```
s1 <- read.pdb( "4AKE" )  
  
s1.chainA <- trim.pdb(s1, chain="A", elety="CA")  
  
s1.b <- s1.chainA$atom$b  
  
plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
```


Input!

```
s1 <- read.pdb("4AKE")  
  
s1.chainA <- trim.pdb(s1, chain="A", elety="CA")  
  
s1.b <- s1.chainA$atom$b  
  
plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
```

pdb

pdbcode



```
s1 <- read.pdb("4AKE")
```

```
s1.chainA <- trim.pdb(s1, chain="A", elety="CA")
```

```
s1.b <- s1.chainA$atom$b
```

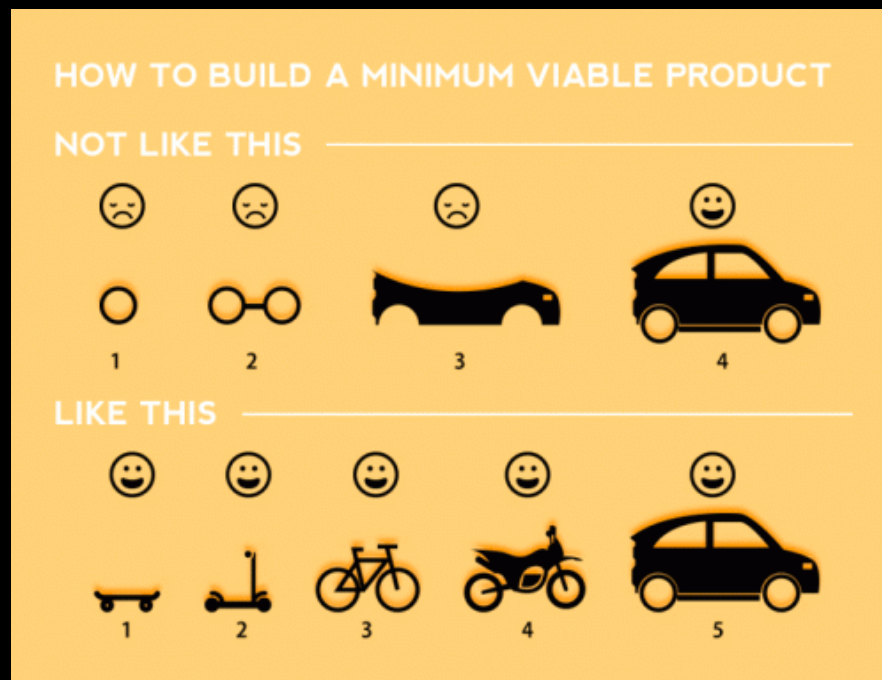
Needed?

```
plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
```

```
pdbcode <- "4AKE"  
  
pdb <- read.pdb(pdbcode)  
  
chain <- trim.pdb(pdb, chain="A", elety="CA")  
  
plotb3(chain$atom$b, sse=chain, typ="l", ylab="Bfactor")
```

```
plotbfactor <- function(pdbcode="4AKE") {  
  
  pdb <- read.pdb(pdbcode)  
  chain <- trim.pdb(pdb, chain="A", elety="CA")  
  
  plotb3(chain$atom$b, sse=chain,  
         typ="l", ylab="Bfactor")  
}
```

Recap...



Build that skateboard before you build the car.

A limited but functional thing is very useful and keeps the spirits high.

[Image credit: Spotify development team]

Suggested steps for writing your functions

- ➔ **How:** Follow a step-by-step procedure to go from working code snippet to refined and tested function.
 1. Start with a simple problem and write a working snippet of code.
 2. Rewrite for clarity and to reduce duplication
 3. Then, and only then, turn into an initial function
 4. Test on small well defined input
 5. Report on potential problem by failing early and loudly!

...



And Now For Something
Completely Different

Video Recap

- Introduction to machine learning
 - Unsupervised, supervised and reinforcement learning
- Clustering
 - K-means clustering
 - Hierarchical clustering
- Dimensionality reduction, visualization and 'structure' analysis
 - Principal Component Analysis (PCA)

Video Recap

- Introduction to machine learning
 - Unsupervised, supervised and reinforcement learning
- Clustering
 - K-means clustering
 - Hierarchical clustering
- Dimensionality reduction, visualization and 'structure' analysis
 - Principal Component Analysis (PCA)

kmeans ()

```
# Generate some example data for clustering
tmp <- c(rnorm(30,-3), rnorm(30,3))
x <- cbind(x=tmp, y=rev(tmp))

plot(x)
```

Use the **kmeans()** function setting k to 2 and nstart=20

Inspect/print the results

- Q. How many points are in each cluster?
- Q. What 'component' of your result object details
 - cluster size?
 - cluster assignment/membership?
 - cluster center?

Plot x colored by the kmeans cluster assignment and
add cluster centers as blue points

hclust()

```
# Generate some example data for clustering
tmp <- c(rnorm(30,-3), rnorm(30,3))
x <- cbind(x=tmp, y=rev(tmp))

plot(x)
```

Analyze this same data with **hclust()**

Demonstrate the use of **dist()**, **hclust()**, **plot()**
and **cutree()** functions to do clustering,
Generate dendrograms and return cluster assignment/
membership vector...

Video Recap

- Introduction to machine learning
 - Unsupervised, supervised and reinforcement learning
- Clustering
 - K-means clustering
 - Hierarchical clustering
- Dimensionality reduction, visualization and 'structure' analysis
 - Principal Component Analysis (PCA)

A long time ago in a galaxy far,
far away....



David Robinson

@drob

Following



Every linear algebra class

Me: What are eigenvectors

Teacher: You can think of them as an n -dimensional kernel subspace

Me: No I can't

702 Retweets 1,384 Likes

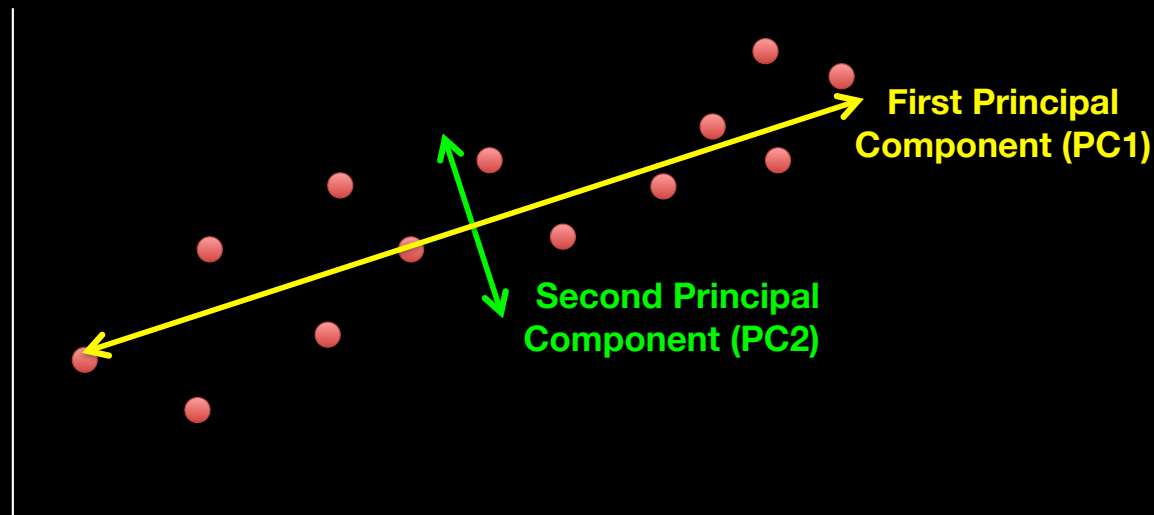


PCA: Principal Component Analysis

PCA projects the features onto the principal components.

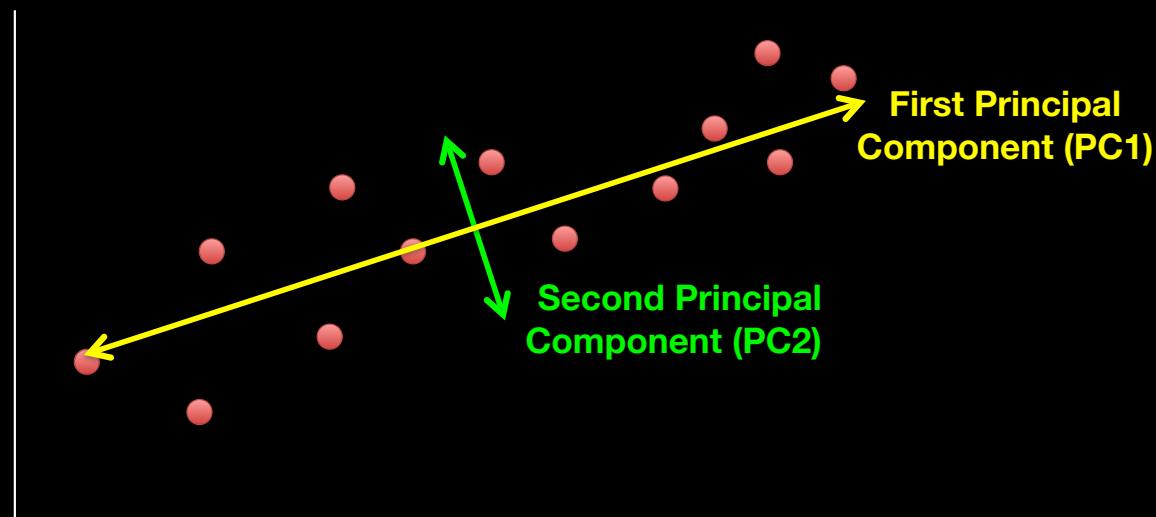
The motivation is to reduce the features dimensionality while only losing a small amount of information.

The first principal component (PC) follows a “best fit” through the data points.



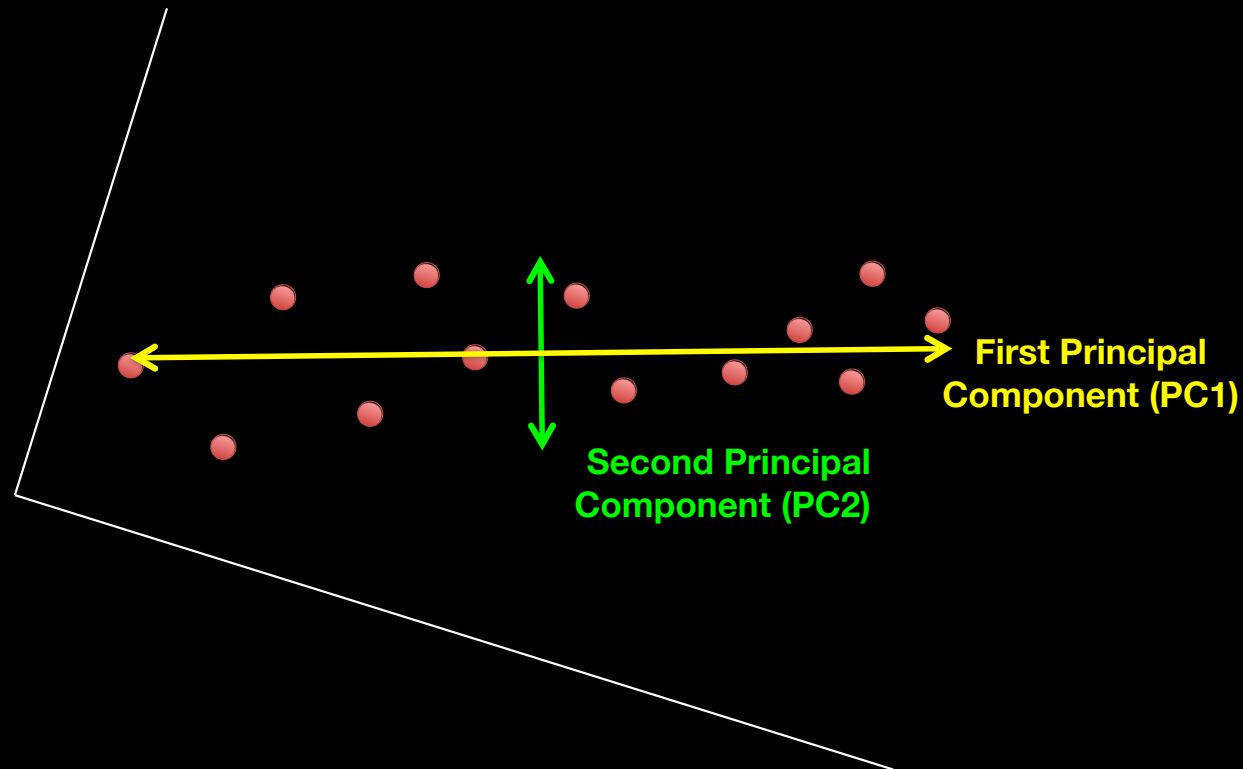
PCA: Principal Component Analysis

Principal components are new low dimensional axis
(or surfaces) closest to the observations



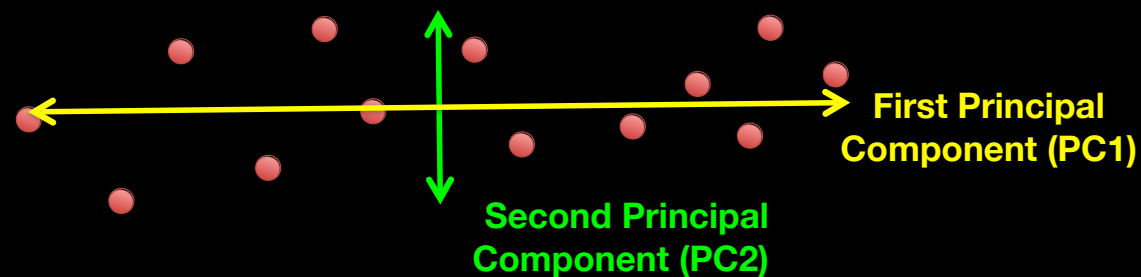
PCA: Principal Component Analysis

Principal components are new low dimensional axis
(or surfaces) closest to the observations



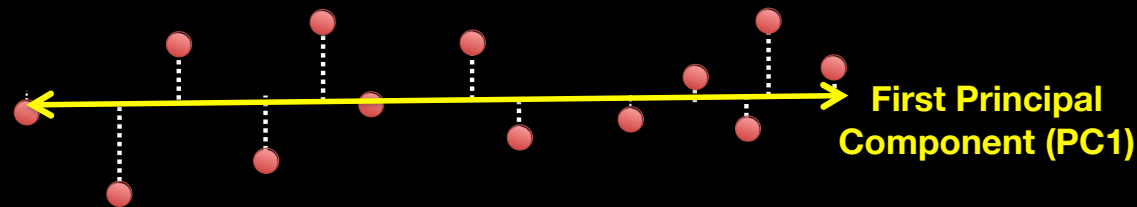
PCA: Principal Component Analysis

Principal components are new low dimensional axis
(or surfaces) closest to the observations



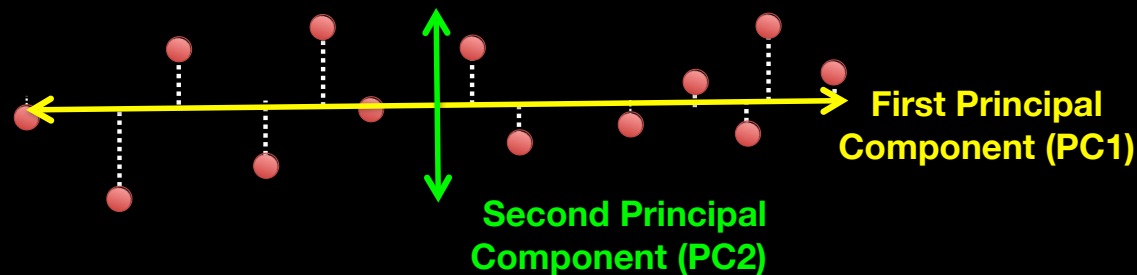
PCA: Principal Component Analysis

The data have maximum variance along **PC1** (then PC2 etc.) which makes the first few PCs useful for visualizing our data and as a basis for further analysis



PCA: Principal Component Analysis

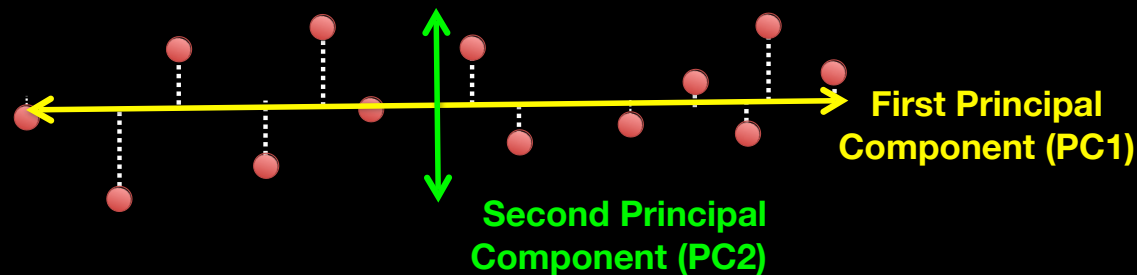
The data have maximum variance along **PC1** (then **PC2** etc.) which makes the first few PCs useful for visualizing our data and as a basis for further analysis



The PCs shown here makes the left/right, above/below variation easier to see.

PCA: Principal Component Analysis

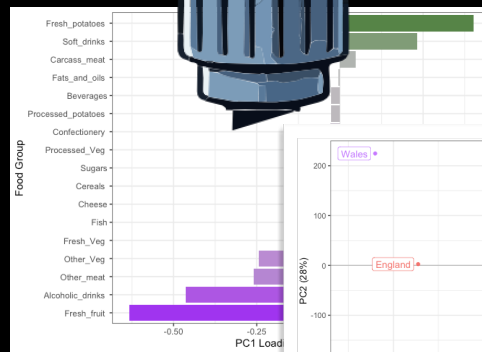
The data have maximum variance along **PC1** (then **PC2** etc.) which makes the first few PCs useful for visualizing our data and as a basis for further analysis



The PCs shown here makes the left/right, above/below variation easier to see.

Recap: PCA objectives

- To reduce dimensionality
- To visualize multidimensional data
- To choose the most useful variables (features)
- To identify groupings of objects (e.g. genes/samples)
- To identify outliers



Your turn!

WebApp

PCA Lab Sheet

Input: read, View/head,

PCA: prcomp,

Cluster: kmeans, hclust

Compare: plot, table, etc.

Reference Slides

```
url <- "https://tinyurl.com/UK-foods"  
food <- read.csv(url, row.names = 1)
```

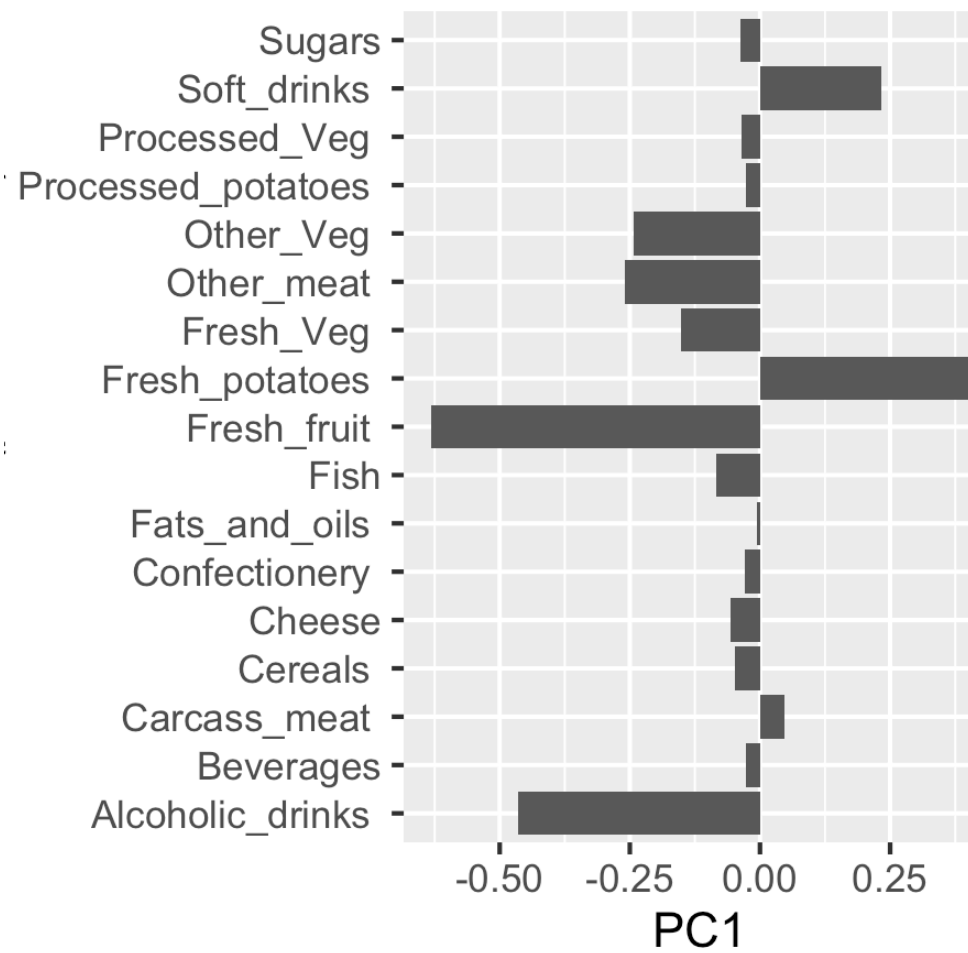
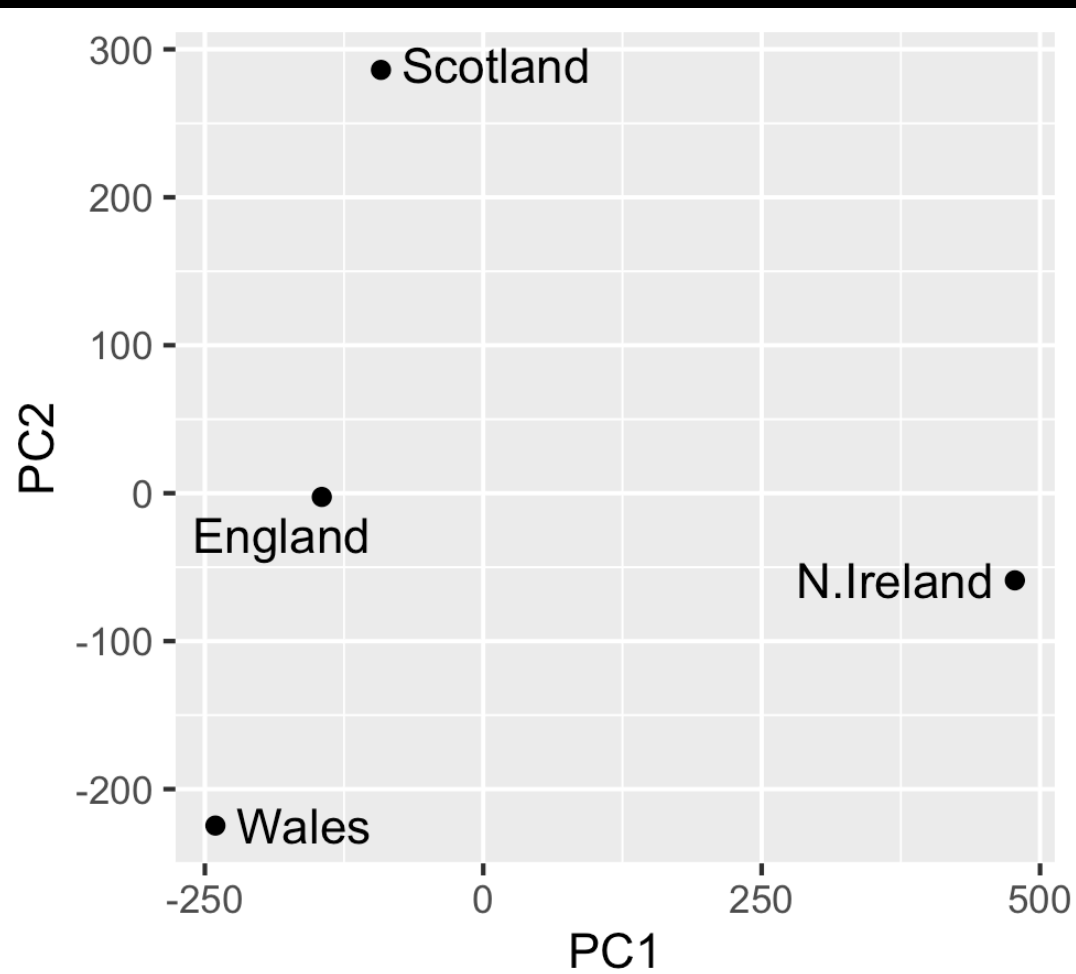
```
url <- "https://tinyurl.com/UK-foods"  
  
food <- read.csv(url, row.names = 1)  
pca <- prcomp( t(food) )
```

```
url <- "https://tinyurl.com/UK-foods"

food <- read.csv(url, row.names = 1)
pca <- prcomp( t(food) )

ggplot(pca$x) +
  aes(PC1, PC2, label=rownames(pca$x)) +
  geom_point() +
  geom_text_repel()
```

```
url <- "https://tinyurl.com/UK-foods"  
  
food <- read.csv(url, row.names = 1)  
pca <- prcomp( t(food) )  
  
ggplot(pca$rotation) +  
  aes(PC1, rownames(pca$rotation)) +  
  geom_col()
```



Practical PCA issue:

Scaling

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

Practical PCA issue:

Scaling

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Means and standard deviations vary a lot
```

```
> round(colMeans(mtcars), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
20.09	6.19	230.72	146.69	3.60	3.22	17.85	0.44	0.41	3.69	2.81

Practical PCA issue:

Scaling

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Means and standard deviations vary a lot
```

```
> round(colMeans(mtcars), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
20.09	6.19	230.72	146.69	3.60	3.22	17.85	0.44	0.41	3.69	2.81

Practical PCA issue:

Scaling

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Means and standard deviations vary a lot
```

```
> round(colMeans(mtcars), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
20.09	6.19	230.72	146.69	3.60	3.22	17.85	0.44	0.41	3.69	2.81

```
> round(apply(mtcars, 2, sd), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
6.03	1.79	123.94	68.56	0.53	0.98	1.79	0.50	0.50	0.74	1.62

Practical PCA issue:

Scaling

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Means and standard deviations vary a lot
```

```
> round(colMeans(mtcars), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
20.09	6.19	230.72	146.69	3.60	3.22	17.85	0.44	0.41	3.69	2.81

```
> round(apply(mtcars, 2, sd), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
6.03	1.79	123.94	68.56	0.53	0.98	1.79	0.50	0.50	0.74	1.62

Practical PCA issue:

Scaling

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Means and standard deviations vary a lot
```

```
> round(colMeans(mtcars), 2)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
20.09	6.19	230.72	146.69	3.60	3.22	17.85	0.44	0.41	3.69	2.81

```
> round(apply(mtcars, 2, sd), 2)
```

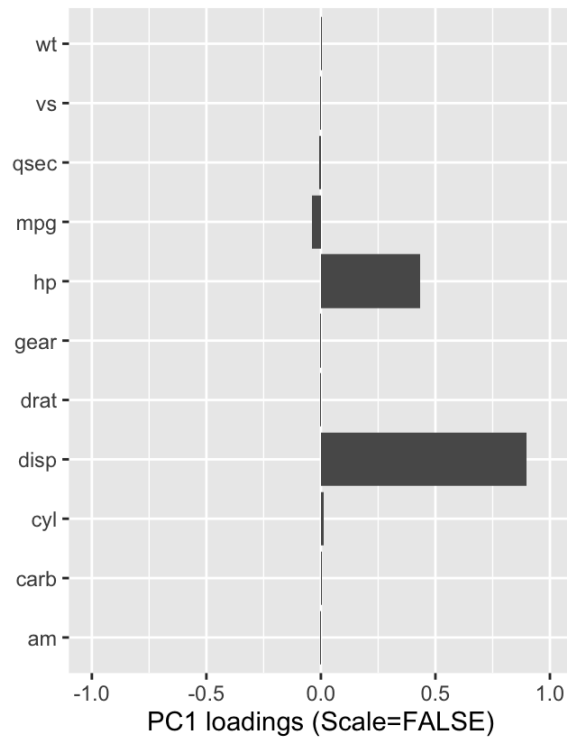
mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
6.03	1.79	123.94	68.56	0.53	0.98	1.79	0.50	0.50	0.74	1.62

In general we want to **scale and center** our data prior to PCA to ensure that each feature contributes equally to the analysis, preventing variables with larger scales from dominating the principal components.

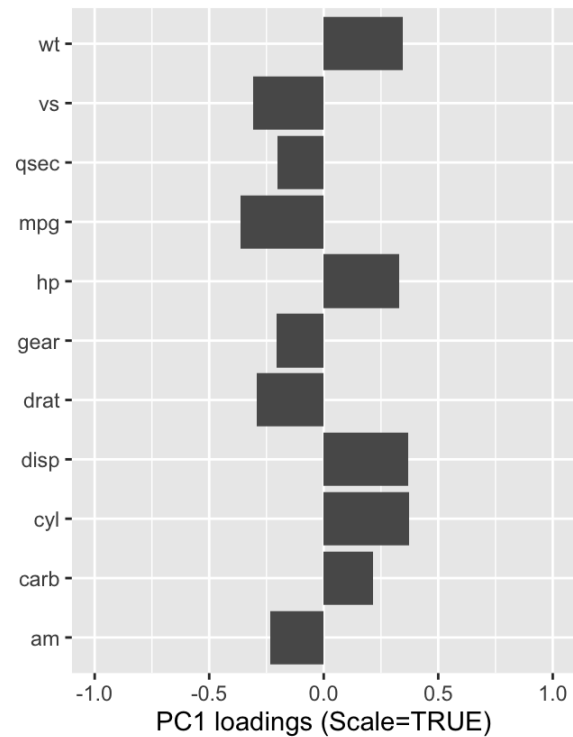
Practical PCA issue:

Scaling

```
prcomp(x, scale=FALSE)
```

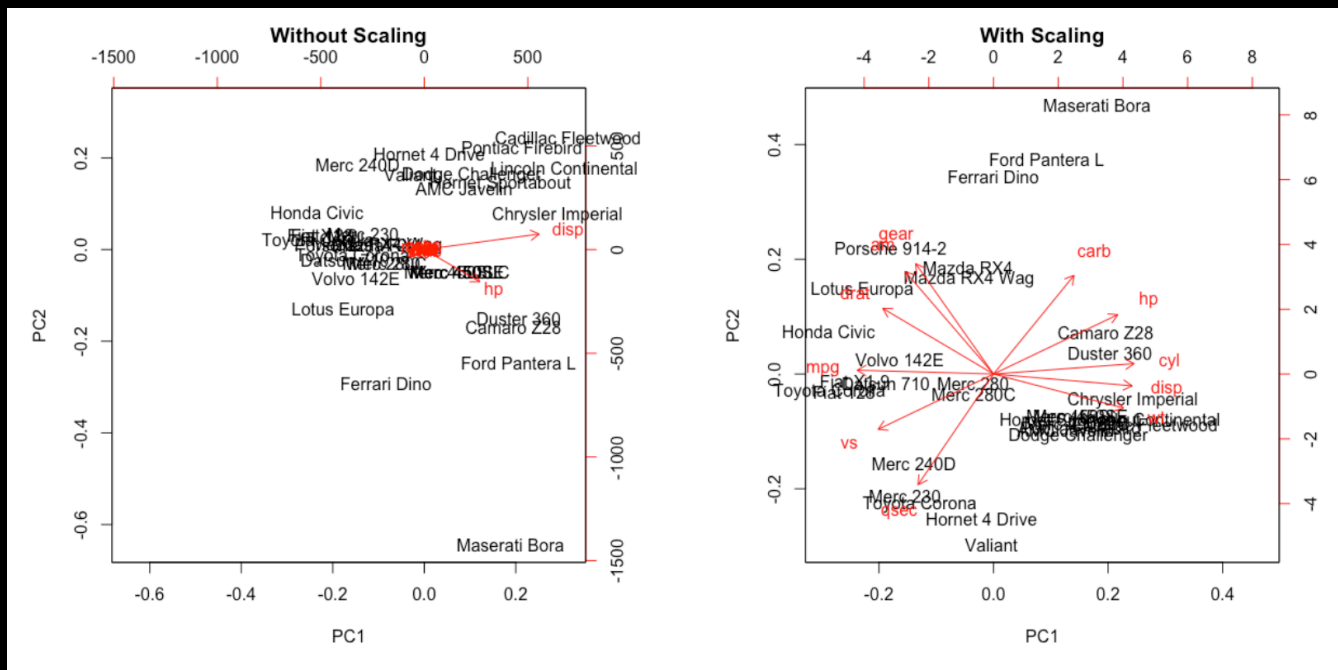


```
prcomp(x, scale=TRUE)
```



Practical PCA issue: Scaling

```
prcomp(x, scale=FALSE)
```



```
prcomp(x, scale=TRUE)
```

Following slides adapted from the excellent

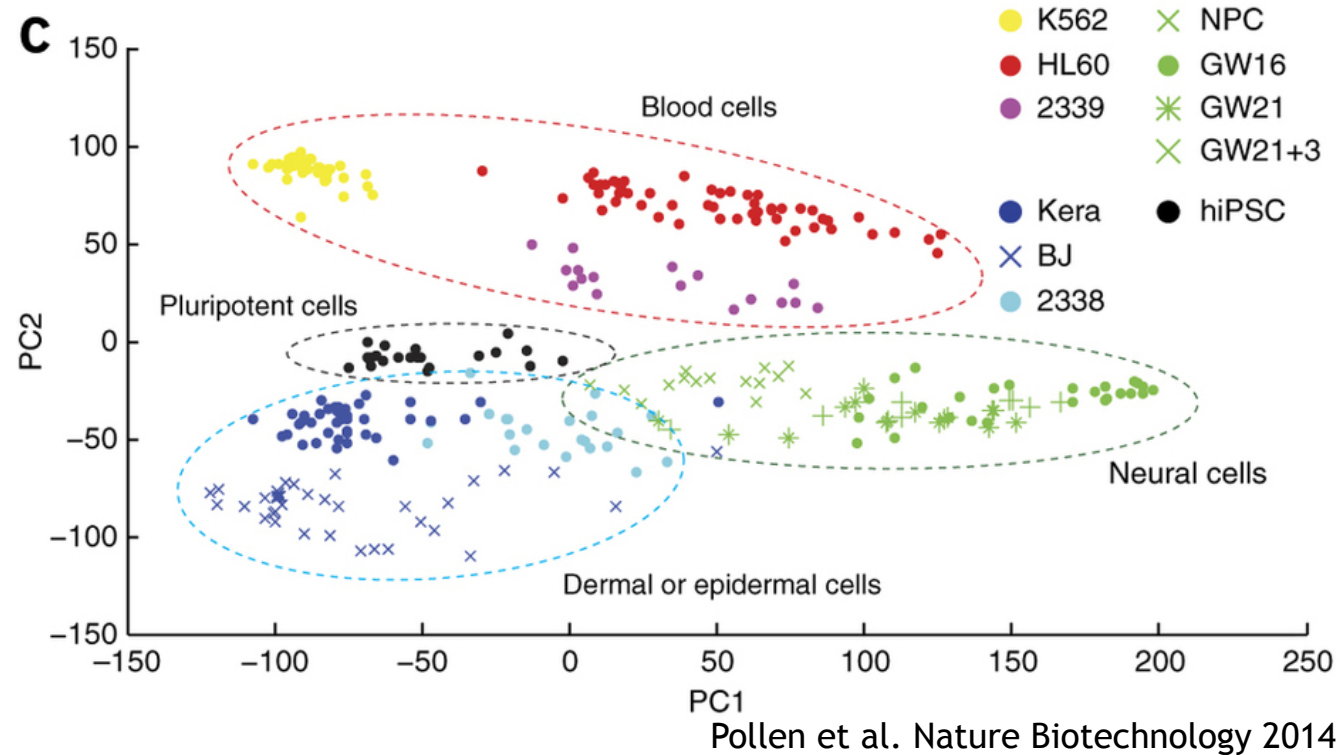
StatQuest: PCA Clearly Explained

By Joshua Starmer

[https://youtu.be/ UVHneBUBW0](https://youtu.be/UVHneBUBW0)

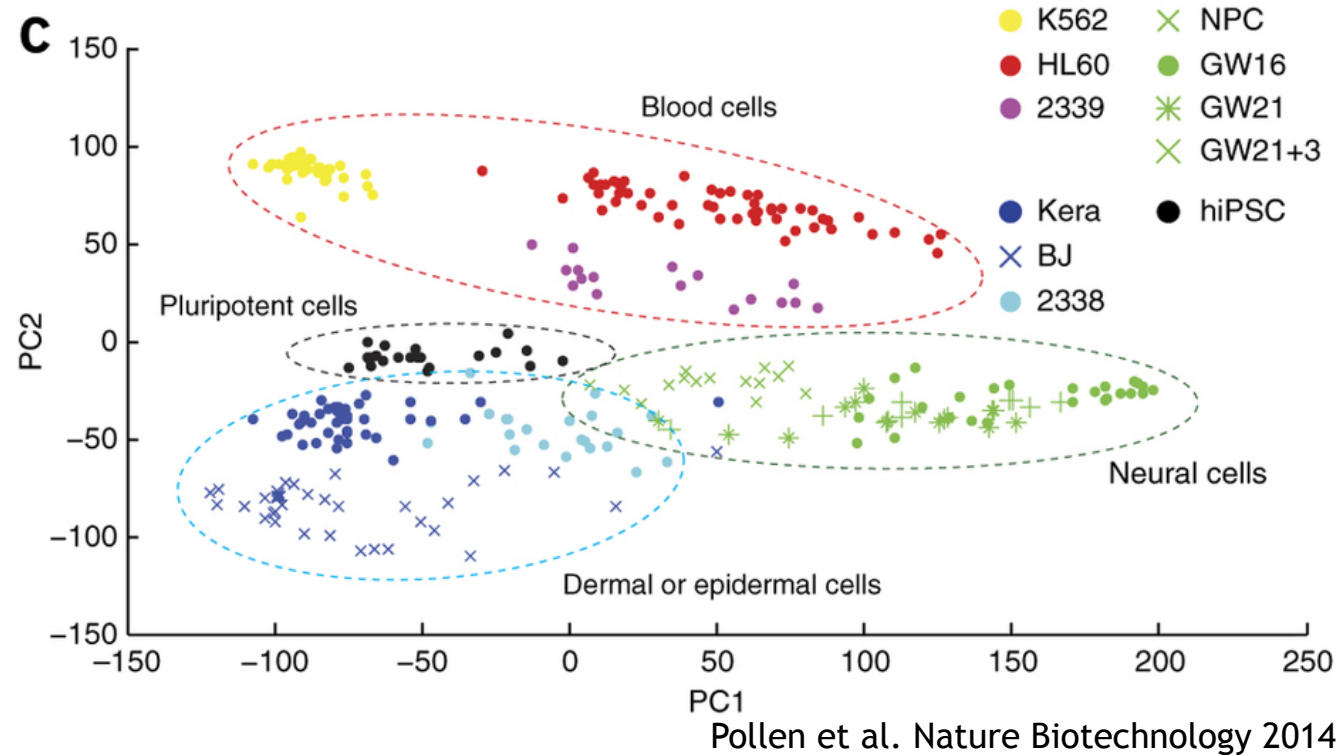
This PCA plot shows clusters of cell types.

This graph was drawn from single-cell RNA-seq.
There were about 10,000 transcribed genes in each cell.



This PCA plot shows clusters of cell types.

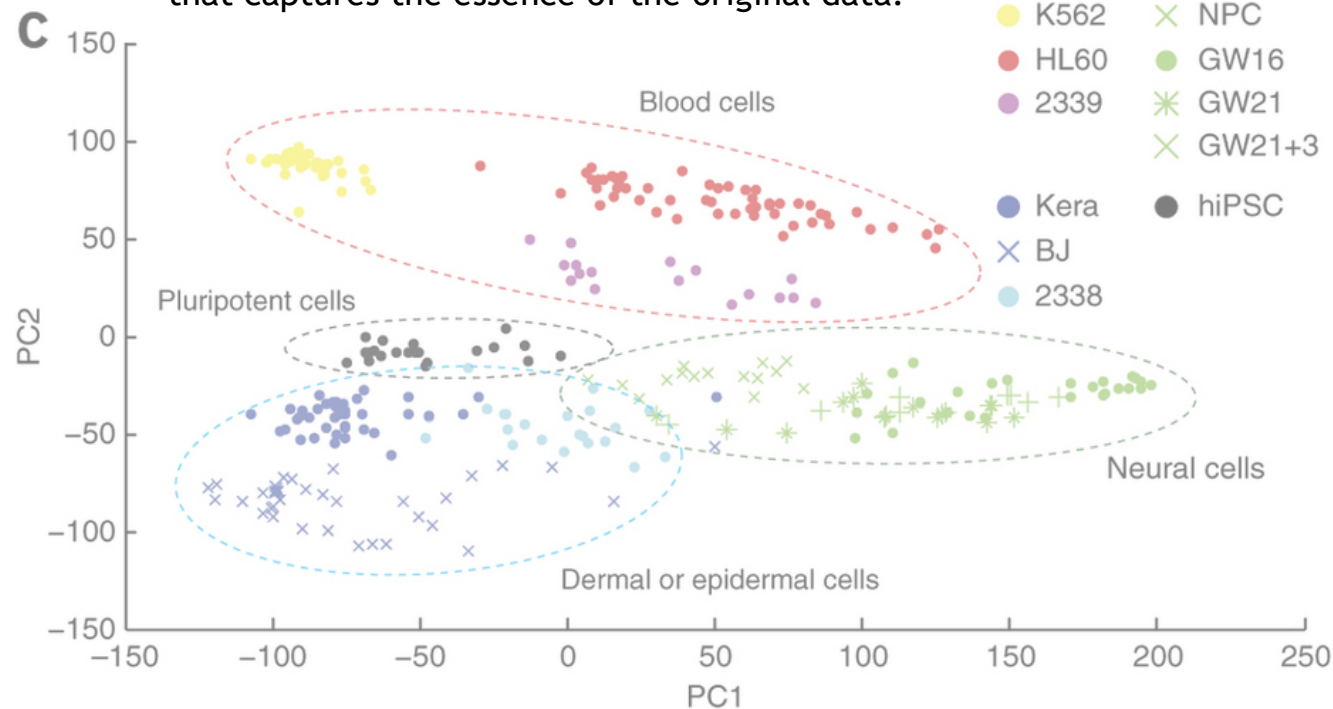
Each dot represents a single-cell and its transcription profile
The general idea is that cells with similar transcription should cluster.



This PCA plot shows clusters of cell types.

How does transcription from 10,000 genes get compressed to a single dot on a graph?

PCA is a method for compressing a lot of data into something that captures the essence of the original data.



What does PCA aim to do?

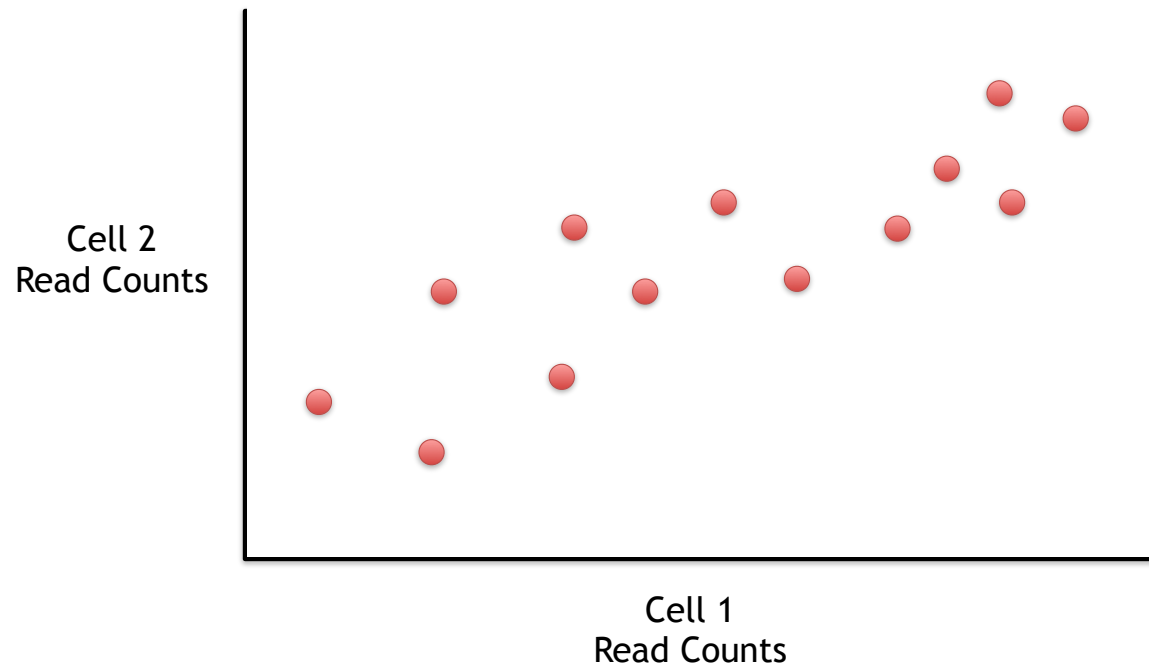
- PCA takes a dataset with a lot of dimensions (i.e. lots of cells) and flattens it to 2 or 3 dimensions so we can look at it.
 - It tries to find a meaningful way to flatten the data by focusing on the things that are different between cells. (much, much more on this later)

A PCA example

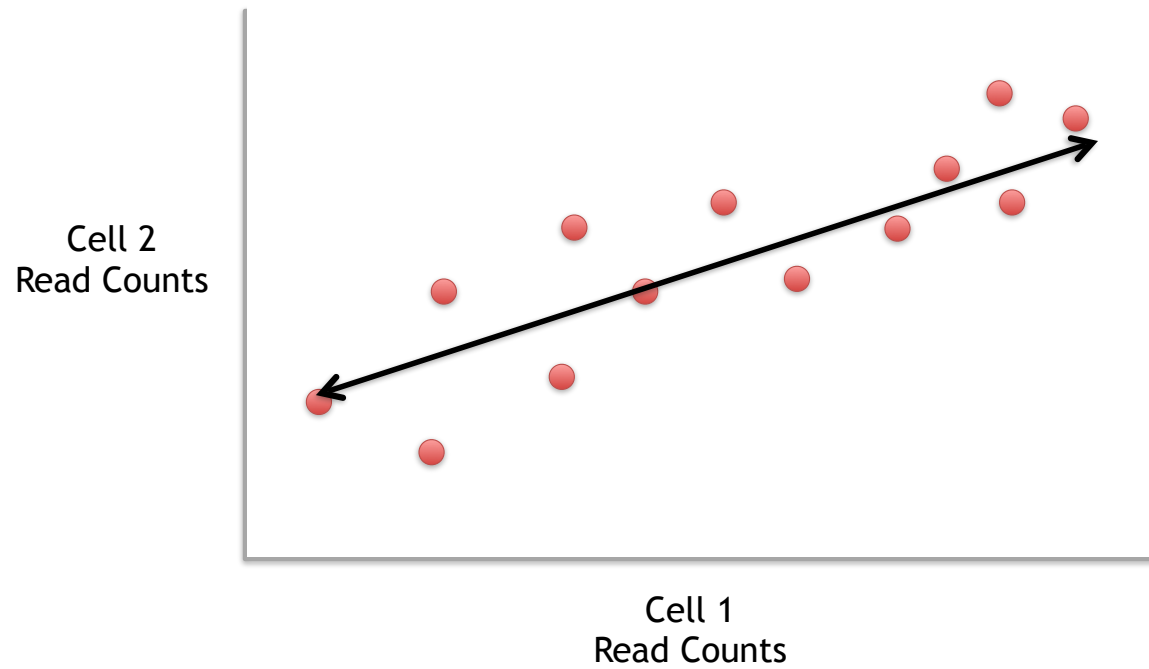
Again, we'll start with just two cells
Here's the data:

Gene	Cell1 reads	Cell2 reads
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
... (etc)	... (etc)	... (etc)

Here is a 2-D plot of the data from 2 cells.

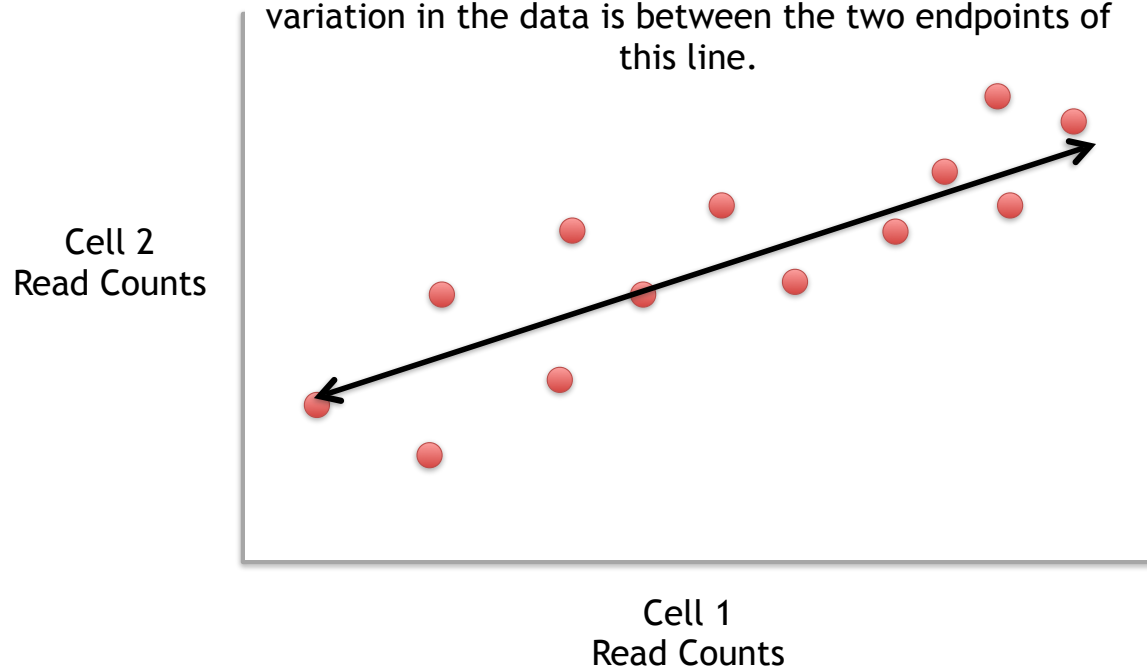


Generally speaking, the dots are spread out along a diagonal line.

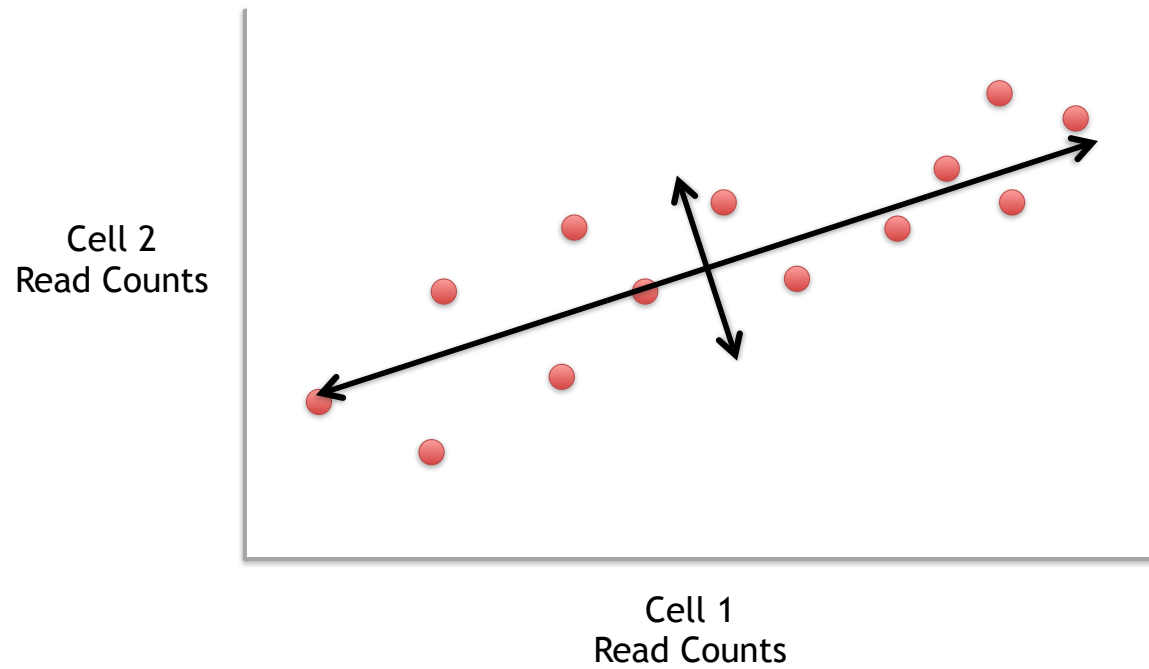


Generally speaking, the dots are spread out along a diagonal line.

Another way to think about this is that the maximum variation in the data is between the two endpoints of this line.

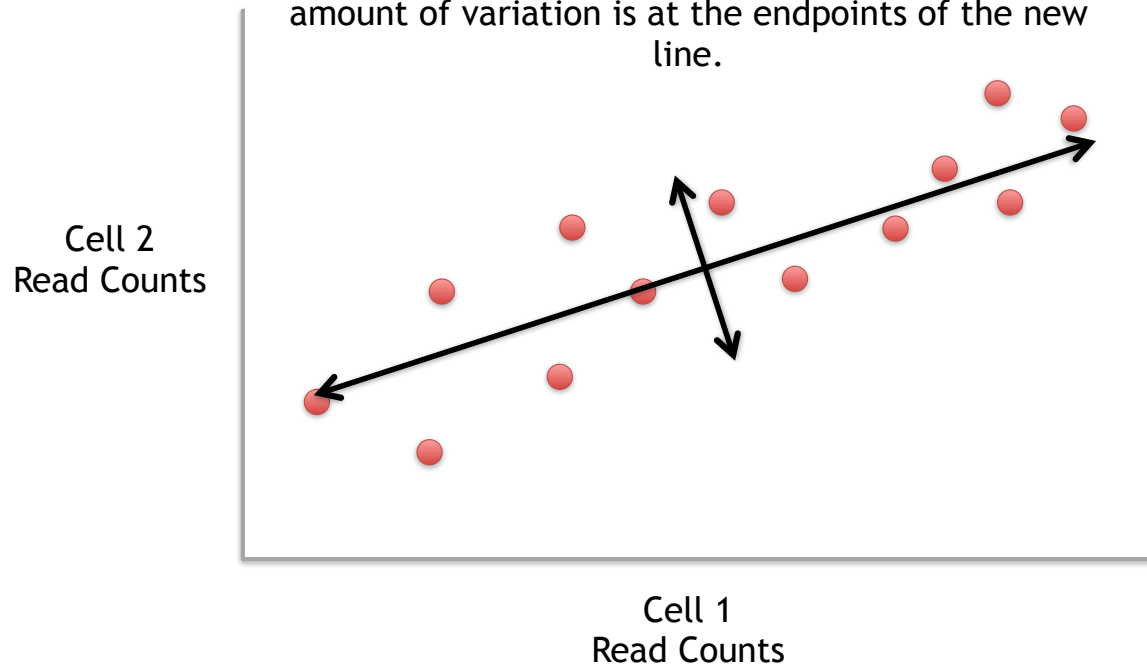


Generally speaking, the dots are also spread out a little above and below the first line.

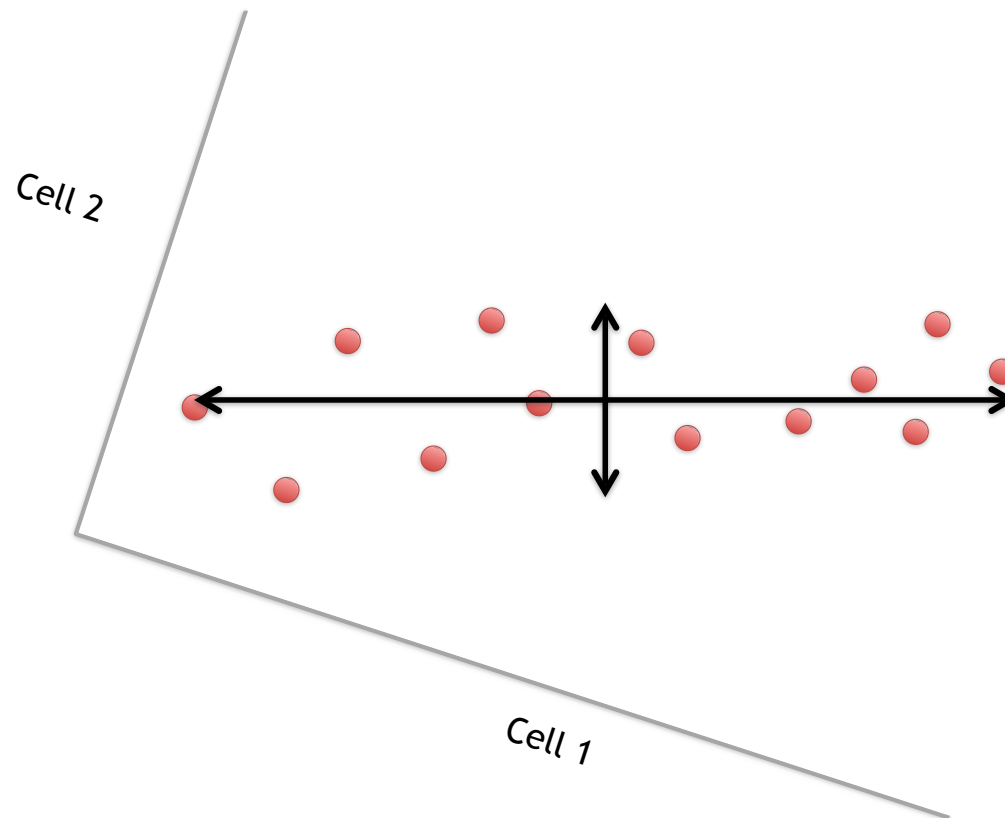


Generally speaking, the dots are also spread out a little above and below the first line.

Another way to think about this is that the 2nd largest amount of variation is at the endpoints of the new line.

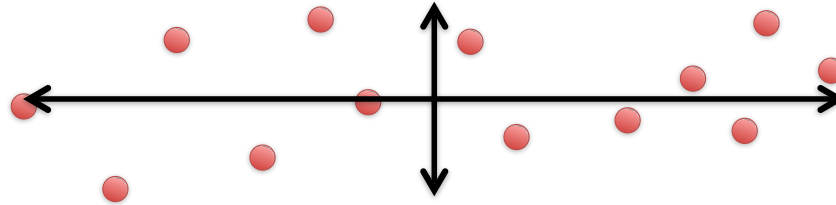


If we rotate the whole graph, the two lines that we drew make new X and Y axes.



If we rotate the whole graph, the two lines that we drew make new X and Y axes.

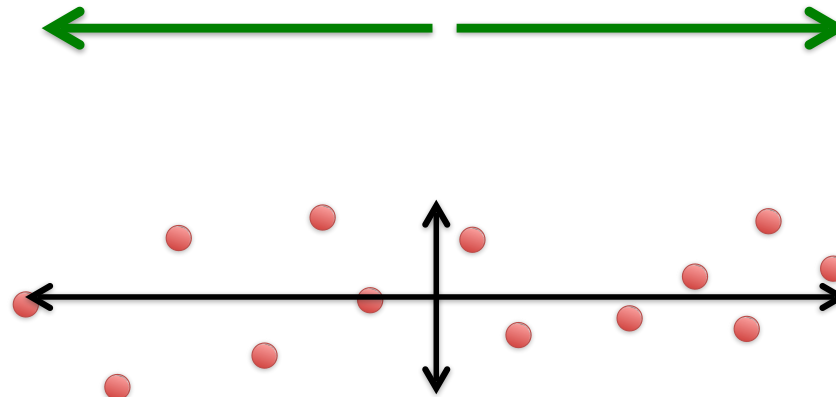
This makes the left/right, above/below variation easier to see.



If we rotate the whole graph, the two lines that we drew make new X and Y axes.

This makes the left/right, above/below variation easier to see.

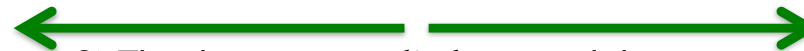
- 1) The data varies **a lot** left and right



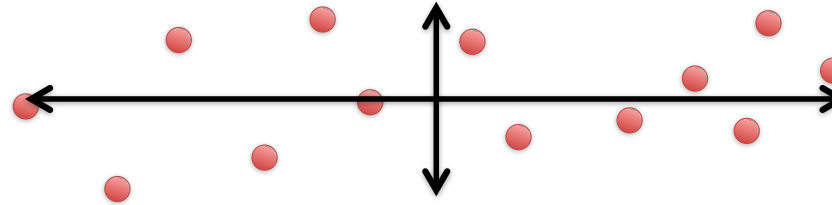
If we rotate the whole graph, the two lines that we drew make new X and Y axes.

This makes the left/right, above/below variation easier to see.

- 1) The data varies **a lot** left and right



- 2) The data varies **a little** up and down



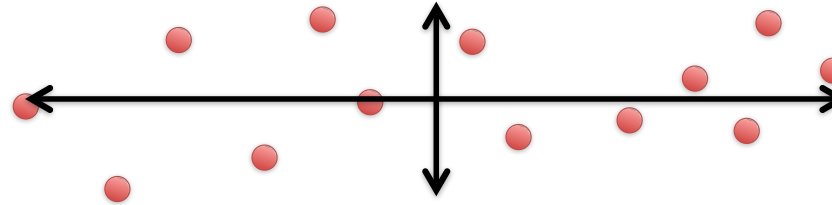
If we rotate the whole graph, the two lines that we drew make new X and Y axes.

This makes the left/right, above/below variation easier to see.

1) The data varies **a lot** left and right



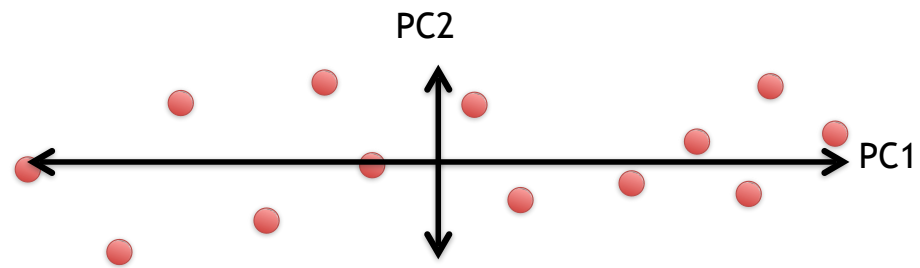
2) The data varies **a little** up and down



Note: All of the points can be drawn in terms of left/right + up/down, just like any other 2-D graph.

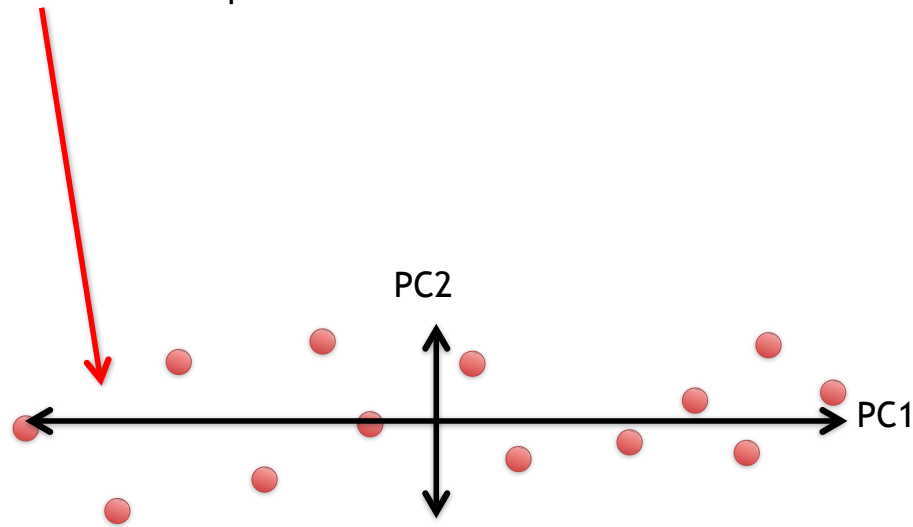
That is to say, we do not need another line to describe “diagonal” variation - we’ve already captured the two directions that can have variation.

These two “new” (or “rotated”) axes that describe the variation in the data are “Principal Components” (PCs)



These two “new” axes that describe the variation in the data are “Principal Components” (PCs)

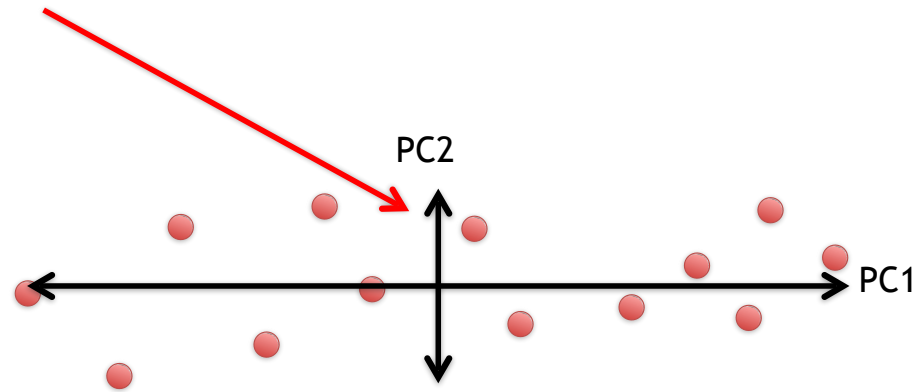
PC1 (the first principal component) is the axis that spans the most variation.



These two “new” axes that describe the variation in the data are “Principal Components” (PCs)

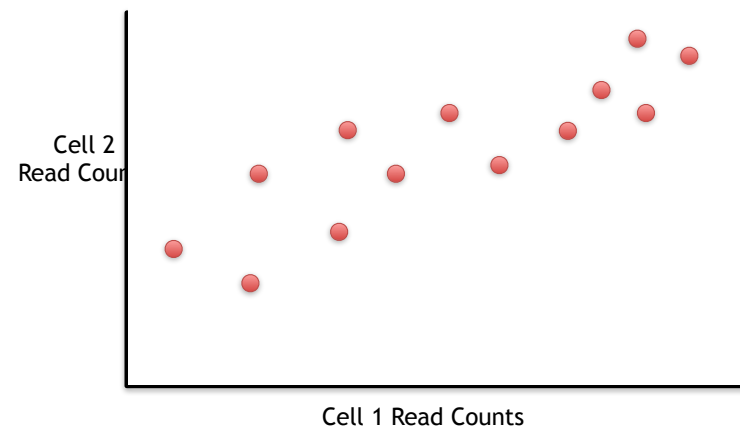
PC1 (the first principal component) is the axis that spans the most variation.

PC2 is the axis that spans the second most variation.



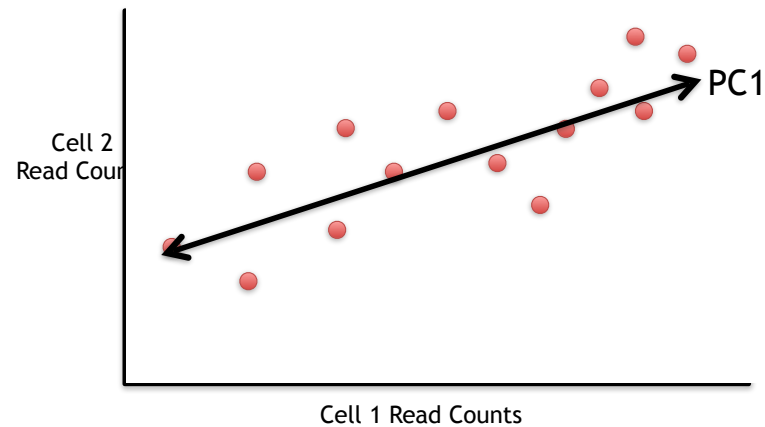
General ideas so far...

- For each gene, we plotted a point based on how many reads were from each cell.



General ideas so far...

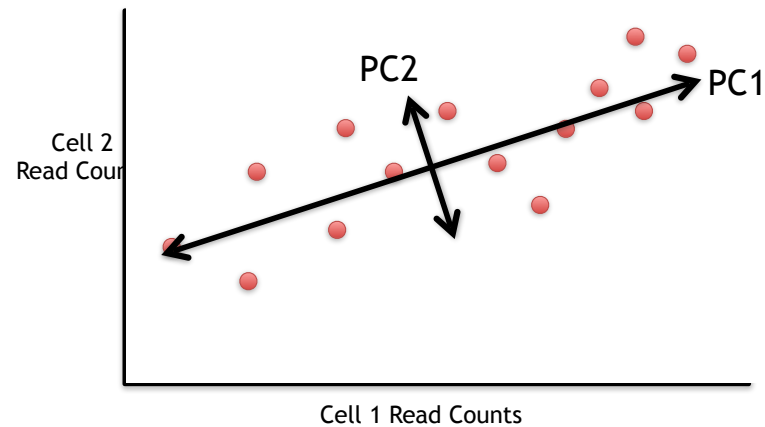
- For each gene, we plotted a point based on how many reads were from each cell.



- PC1 captures the direction where most of the variation is.

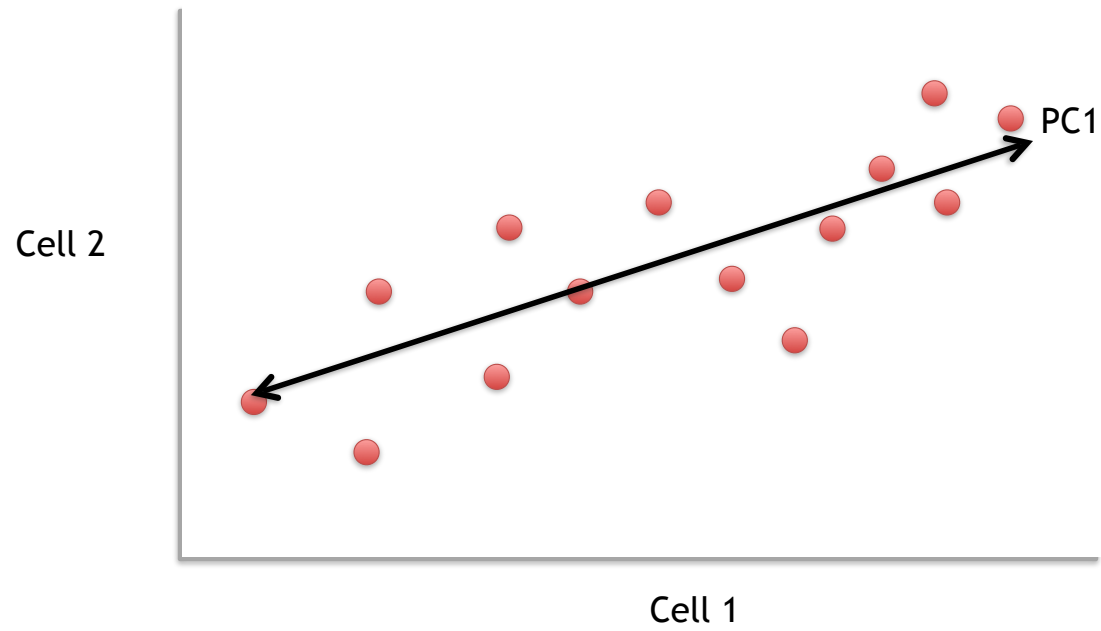
General ideas so far...

- For each gene, we plotted a point based on how many reads were from each cell.

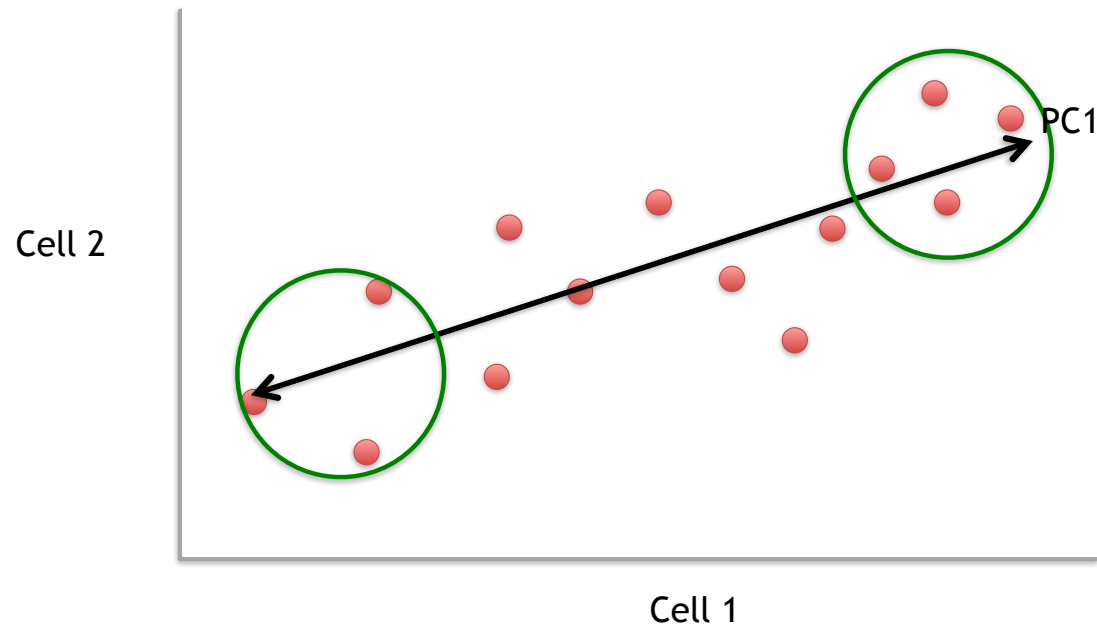


- PC1 captures the direction where most of the variation is.
- PC2 captures the direction with the 2nd most variation.

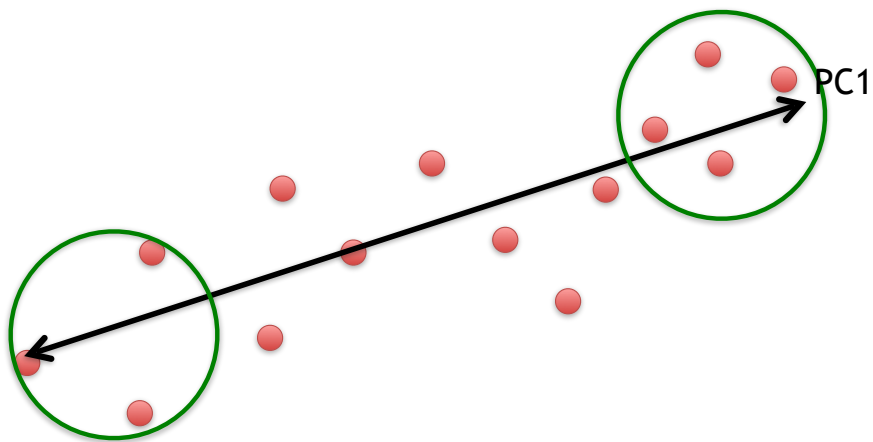
For now, let's focus on PC1



The length and direction of PC1 is mostly determined by the circled genes.

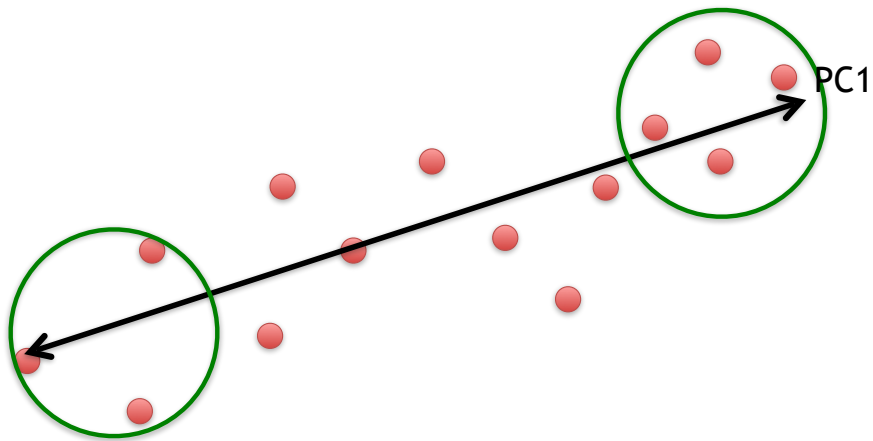


The length and direction of PC1 is mostly determined by the circled genes.

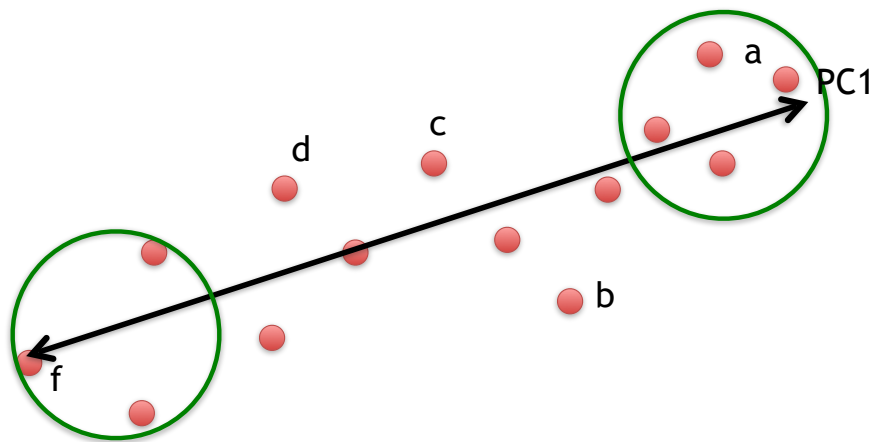


The length and direction of PC1 is mostly determined by the circled genes.

We can score genes based on how much they influence PC1.



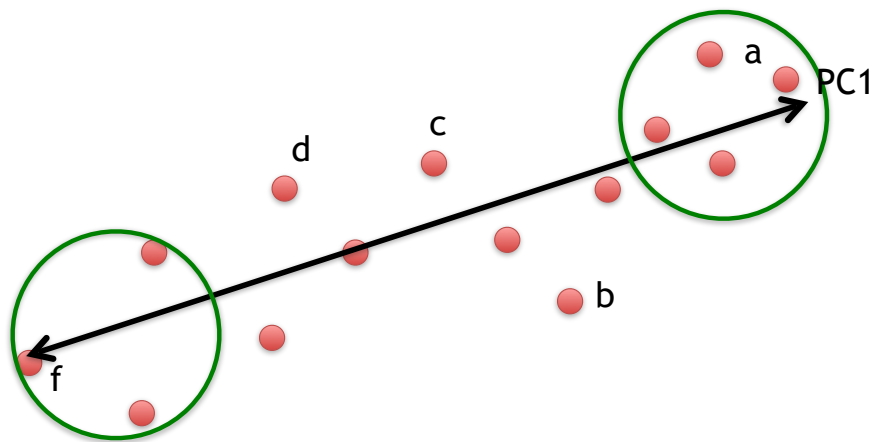
The length and direction of PC1 is mostly determined by the circled genes.



We can score genes based on how much they influence PC1.

Gene	Influence on PC1
a	high
b	low
c	low
d	low
e	high
f	high
...	...

The length and direction of PC1 is mostly determined by the circled genes.

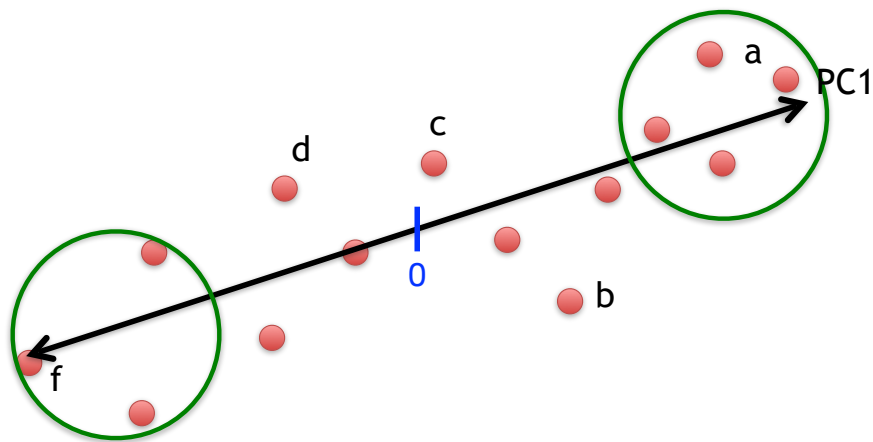


Some genes have more influence on PC1 than others.



Gene	Influence on PC1
a	high
b	low
c	low
d	low
e	high
f	high
...	...

The length and direction of PC1 is mostly determined by the circled genes.



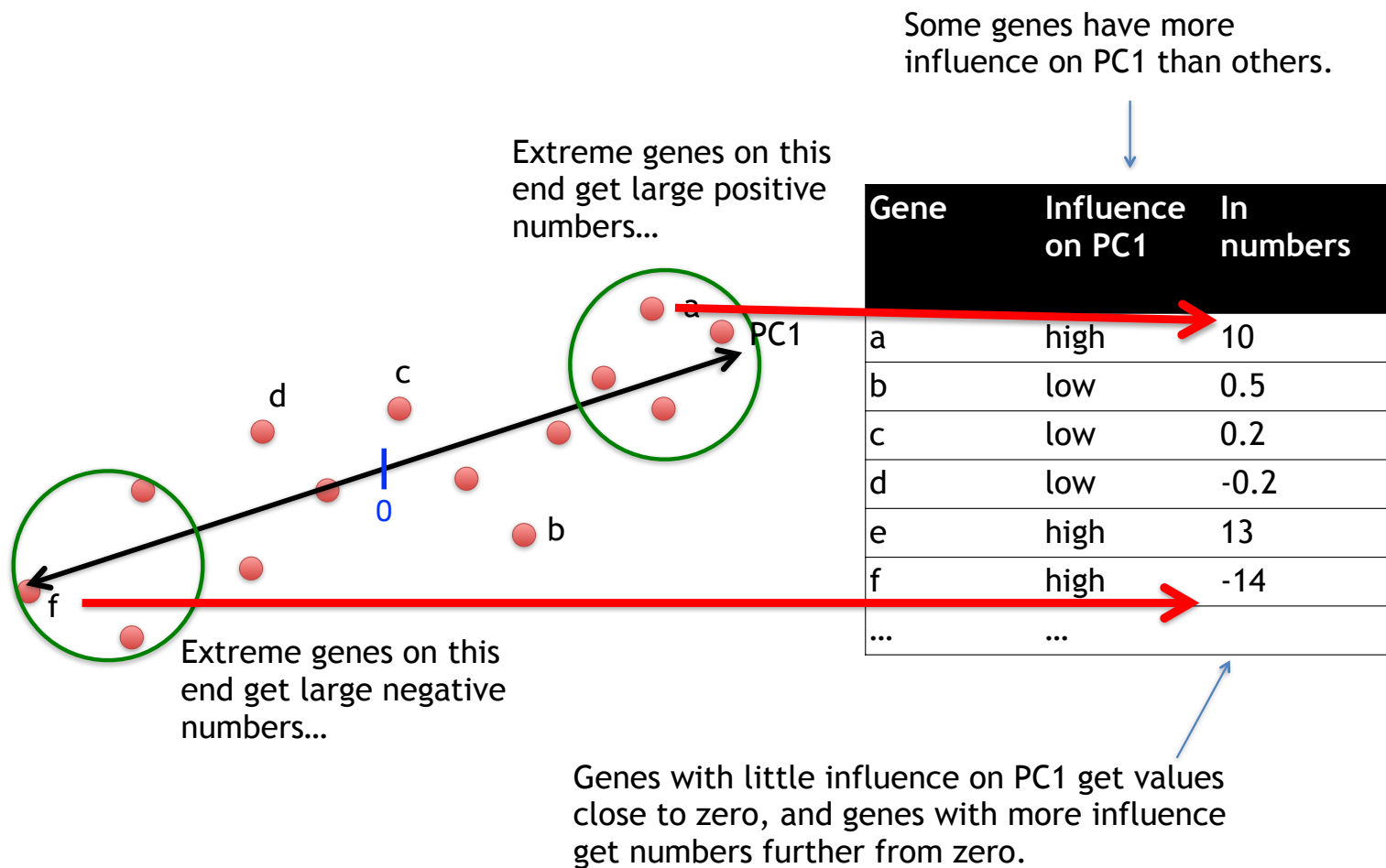
Some genes have more influence on PC1 than others.



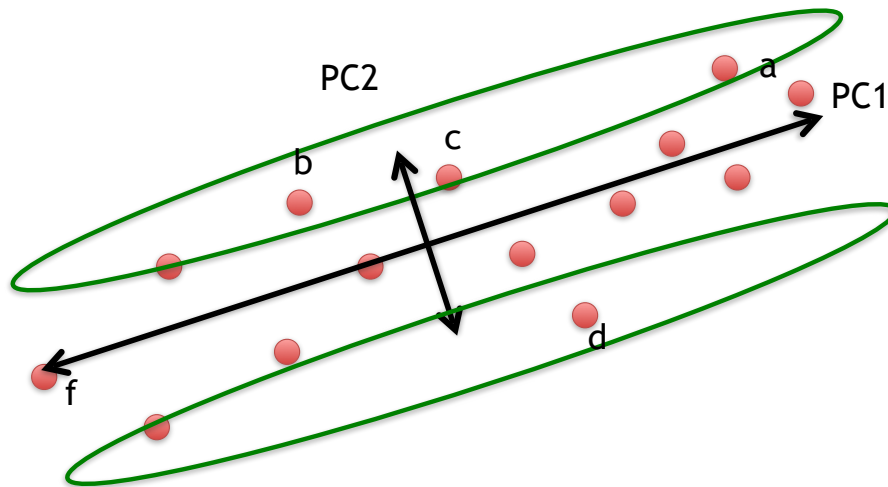
Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	...



Genes with little influence on PC1 get values close to zero, and genes with more influence get numbers further from zero.



Genes that influence PC2



Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

Our two PCs

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	...

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	...

Cell1 PC1 score = (read count * influence) + ... for all genes

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	...

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	...

Cell1 PC1 score = $(10 * 10) + \dots$

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

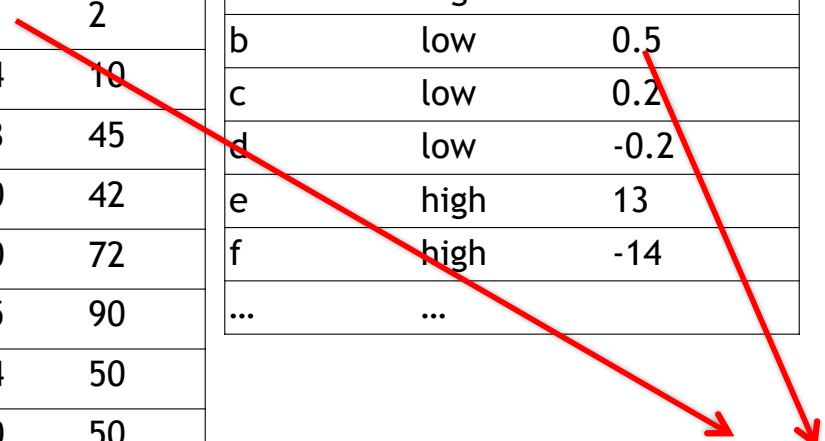
Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	



Cell1 PC1 score = $(10 * 10) + (0 * 0.5) + \dots$

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

$$\text{Cell1 PC1 score} = (10 * 10) + (0 * 0.5) + \dots \text{etc...} = 12$$

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

Cell1 PC1 score = $(10 * 10) + (0 * 0.5) + \dots$ etc... = 12

Cell1 PC2 score = $(10 * 3) + \dots$

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

Cell1 PC1 score = $(10 * 10) + (0 * 0.5) + \dots$ etc... = 12

Cell1 PC2 score = $(10 * 3) + (0 * 10) + \dots$

Using the two Principal Components to plot cells

Combining the read counts for all genes in a cell to get a single value.

The original read counts

Gene	Cell1	Cell2
a	10	8
b	0	2
c	14	10
d	33	45
e	50	42
f	80	72
g	95	90
h	44	50
i	60	50
etc	etc	etc

PC1

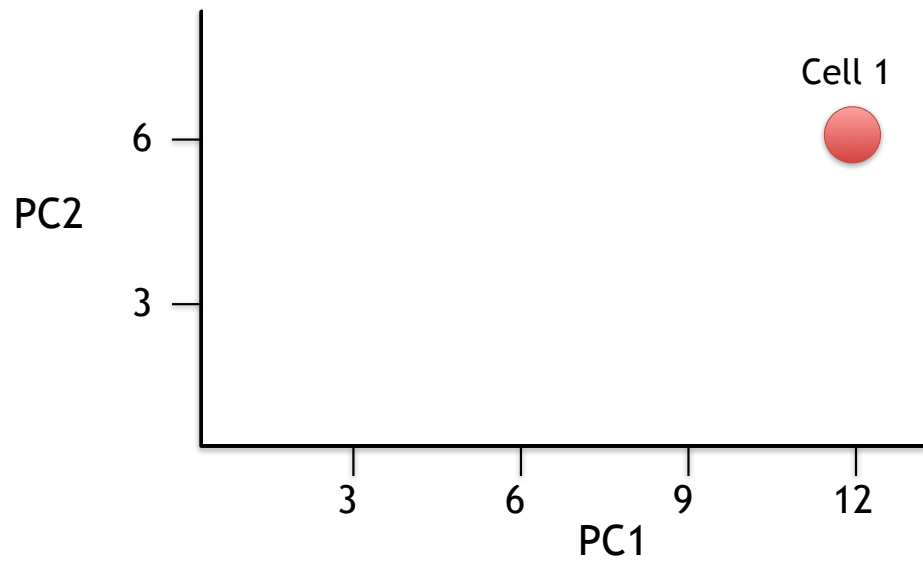
Gene	Influence on PC1	In numbers
a	high	10
b	low	0.5
c	low	0.2
d	low	-0.2
e	high	13
f	high	-14
...	...	

PC2

Gene	Influence on PC2	In numbers
a	medium	3
b	high	10
c	high	8
d	high	-12
e	low	0.2
f	low	-0.1
...	...	

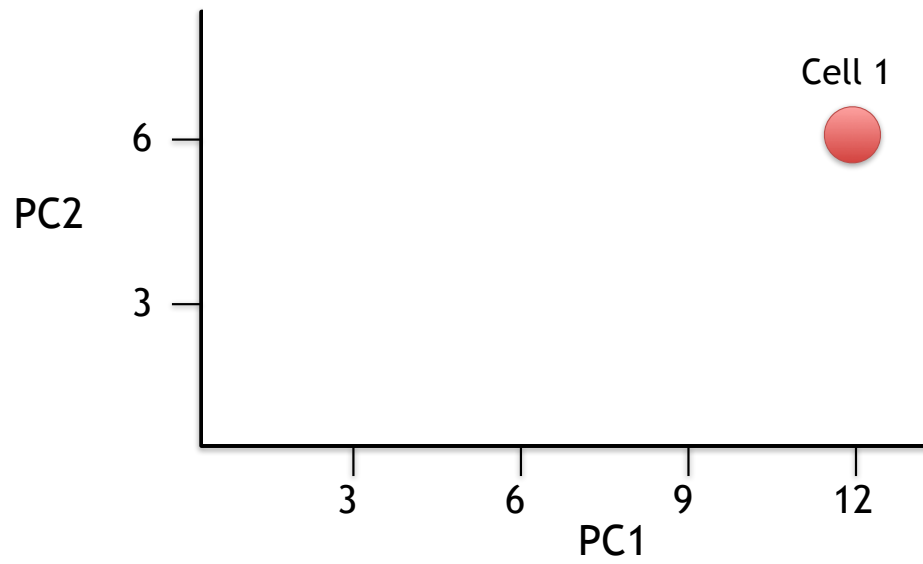
$$\text{Cell1 PC1 score} = (10 * 10) + (0 * 0.5) + \dots \text{etc...} = 12$$

$$\text{Cell1 PC2 score} = (10 * 3) + (0 * 10) + \dots \text{etc...} = 6$$

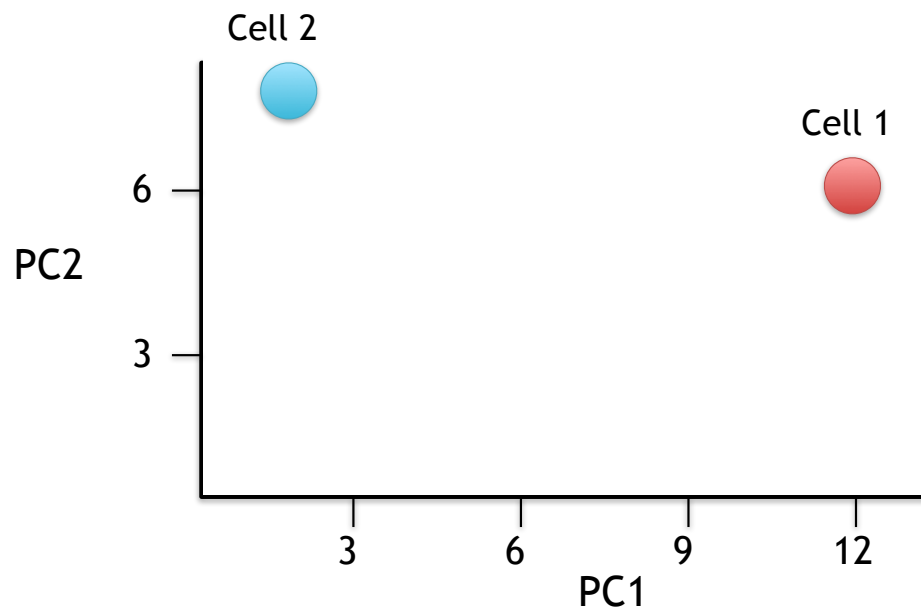


Cell1 PC1 score = $(10 * 10) + (0 * 0.5) + \dots \text{etc...} = 12$

Cell1 PC2 score = $(10 * 3) + (0 * 10) + \dots \text{etc...} = 6$



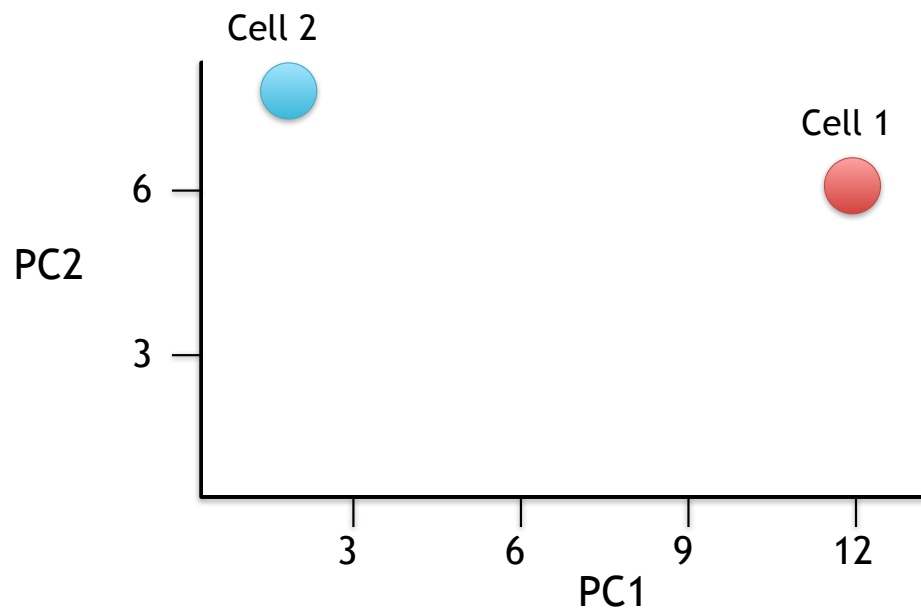
Now calculate scores for Cell2



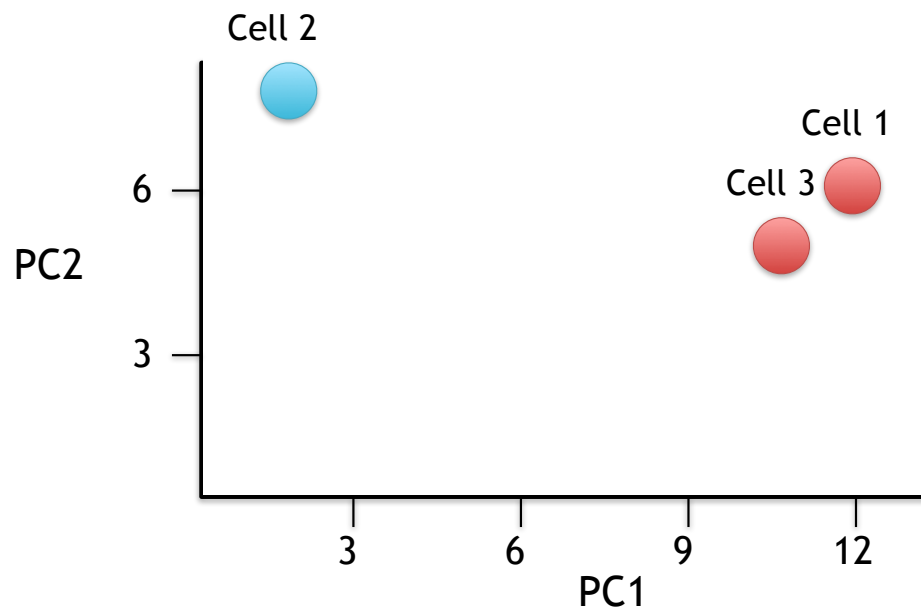
Now calculate scores for Cell2

$$\text{Cell2 PC1 score} = (8 * 10) + (2 * 0.5) + \dots \text{etc...} = 2$$

$$\text{Cell2 PC2 score} = (8 * 3) + (2 * 10) + \dots \text{etc...} = 8$$

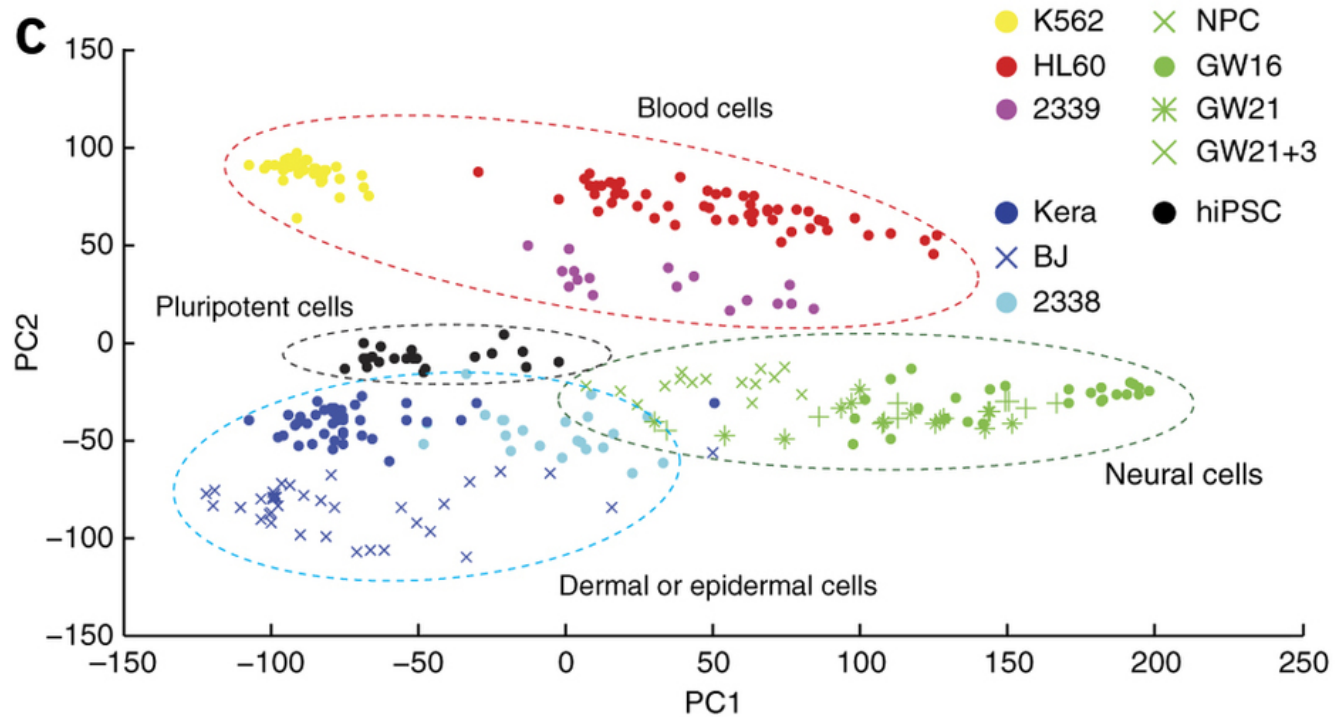


If we sequenced a third cell, and its transcription was similar to cell 1, it would get scores similar to cell 1's.



If we sequenced a third cell, and its transcription was similar to cell 1, it would get scores similar to cell 1's.

Hooray! We know how they plotted all of the cells!!!



Homework:

Unsupervised Learning Mini-Project

Input: read, View/head,

PCA: prcomp,

Cluster: kmeans, hclust

Compare: plot, table, etc.

BONUS: Predictive Modeling with PCA Components

We can use our PCA and clustering models to predict the potential malignancy of new samples:

```
## Predicting Malignancy Of New samples

url <- "https://tinyurl.com/new-samples-CSV"
new <- read.csv(url)
npc <- predict(wisc.pr, newdata=new)

plot(wisc.pr$x[,1:2], col= (diagnosis+1))
points(npc[,1], npc[,2], col="blue", pch=16)
```

[Muddy Point Assessment]